



Otimização de Performance com Memoização no React

Antes de mergulharmos no desafio prático, vamos relembrar os principais conceitos sobre técnicas de memoização para melhorar a performance das suas aplicações React. Revisaremos métodos eficientes para reduzir renderizações desnecessárias e otimizar a experiência do usuário.

Por que usar Memoização?

Memoização é uma técnica usada para **salvar o resultado de uma função** e evitar recálculos desnecessários quando os dados de entrada **permanecem inalterados**. No **React**, isso é fundamental para:

Evitar re-renderizações

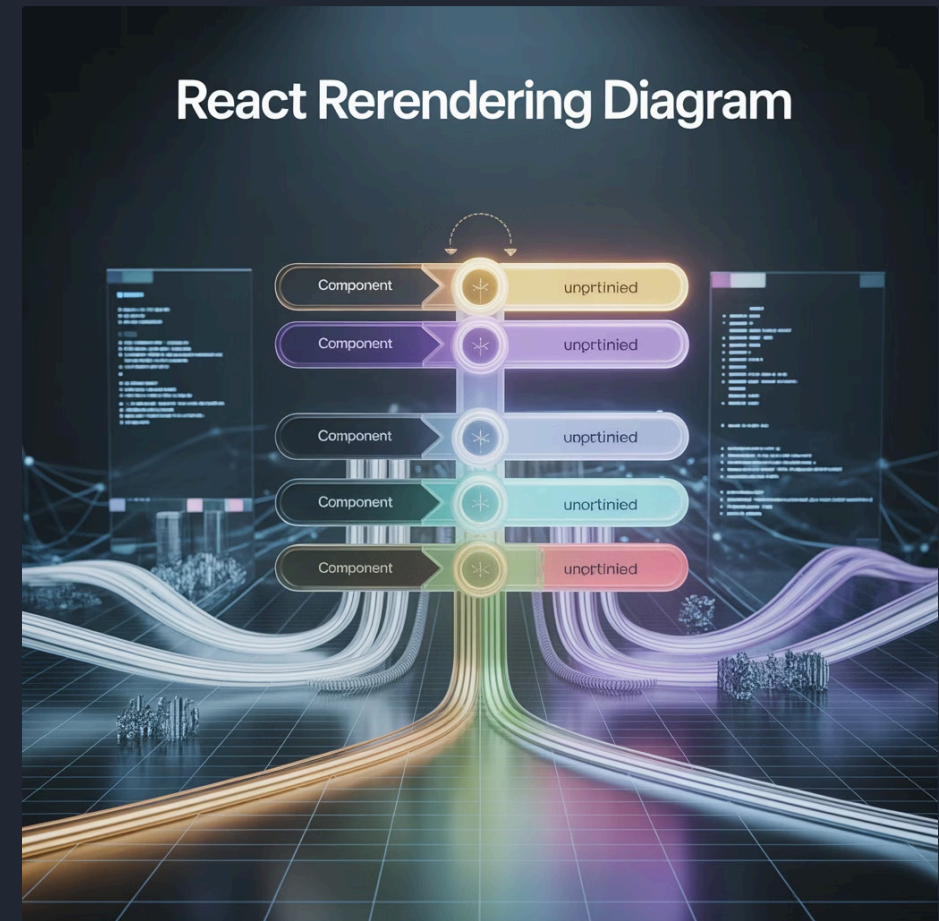
Previne que componentes sejam reconstruídos quando seus dados não mudaram, economizando ciclos de processamento.

Reduzir cálculos pesados

Armazena resultados de operações complexas como filtragens, ordenações e transformações de dados.

Melhorar eficiência

Aumenta a responsividade da interface e reduz o tempo de espera do usuário em aplicações complexas.



⚠️ ⚠️ O uso excessivo pode **aumentar a complexidade do código** sem ganhos significativos, especialmente quando aplicado em componentes leves ou cálculos simples.

Hook: useMemo

O hook `useMemo` é usado para **memoizar valores computados**, ou seja, resultados de funções que só devem ser recalculados quando suas **dependências realmente mudam**.

Casos de uso ideais

- Cálculos computacionalmente intensivos
- Processamento de grandes conjuntos de dados
- Funções que seriam executadas em cada renderização

Implementação

```
const resultado = useMemo(() => {  
  return calculaValor(dados);  
}, [dados]);
```

Só reexecuta se o valor em `dados` mudar. Alterações de referência sem mudança real de conteúdo não disparam recalculações.



Componente: React.memo

O React.memo é utilizado para **evitar re-renderizações de componentes** quando as **props não mudam**. Ele funciona como um "wrapper" que envolve um componente funcional puro.

```
const MyComponent = React.memo(function MyComponentPure({
  name }) {
  return <div>{name}</div>;
});
```

Nesse exemplo, MeuComponentePuro só será re-renderizado se a prop **nome** mudar — evitando processamento desnecessário quando outras partes do estado da aplicação são atualizadas.

1

Componente Original

Re-renderiza sempre que o componente pai atualiza

2

React.memo

Compara props e pula renderização se iguais

3

Resultado

Renderização otimizada e melhor performance

React 19 – Comportamento Automático



A partir do React 19, muitas dessas otimizações já são **aplicadas automaticamente** pelo framework, representando uma evolução significativa no modelo de renderização.

Quando usar memoização manual

- Para forçar otimização em componentes específicos
- Em partes da UI com atualizações muito raras
- Em aplicações legadas que precisam de suporte

Futuro da otimização

- Renderização automática inteligente
- Menor necessidade de otimização manual
- Foco na lógica de negócios ao invés de otimização

- ❑ Mesmo com as melhorias do React 19, entender os princípios de memoização é essencial para desenvolver aplicações React eficientes e escaláveis.